

SmolVLA: Dual-Stream Flow Matching with Frozen Vision-Language Prio

Karthik Goparaju

R&D Intern at Innovarsity and Founder LABKRAZY
goparajukarthik2@gmail.com

Gopi Krishna Ratnakaram

Founder of Innovarsity
info@innovarsity.org

Abstract

Building upon the foundational constraints of purely from-scratch models, SmolVLA introduces pre-trained knowledge distillation by leveraging a frozen OpenAI CLIP ViT-B/32 backbone. This dual-stream architecture processes raw language instructions and camera observations into a unified multimodal context via hierarchical cross-attention layers. The result is a highly parameter-efficient policy network that achieves robust zero-shot generalization while delegating continuous trajectory regression to a lightweight OT-CFM action expert.

1. Architectural Overview

SmolVLA is composed of two primary modules: the pretrained Multimodal Backbone and the Flow-Matching Action Decoder. The backbone utilizes frozen CLIP weights, ensuring that the vast web-scale pre-training is completely preserved without catastrophic forgetting. Images are tokenized into 49 patches, and instructions are parsed into 77 byte-pair encoded tokens. These independent embeddings are projected to a 128-dimensional latent space before fusion.

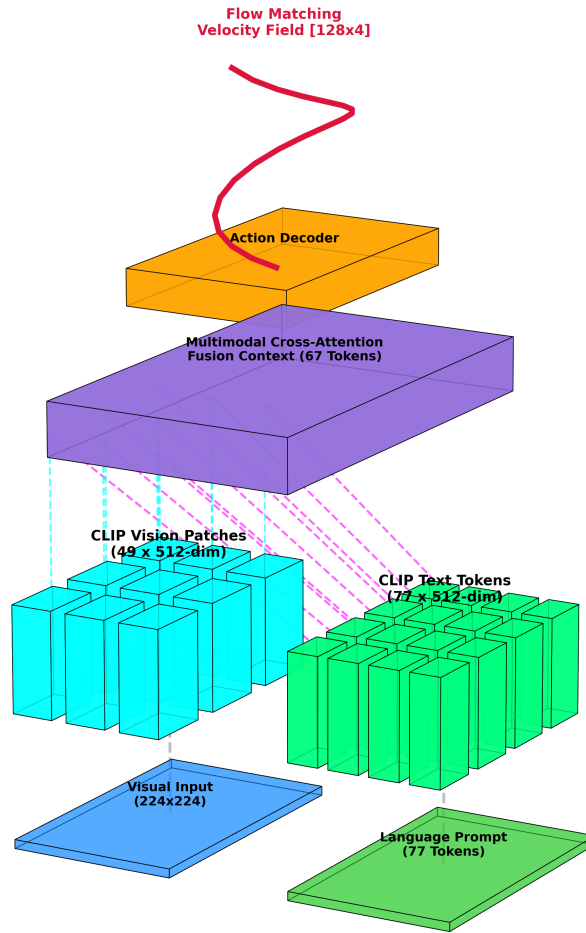


Figure 1: 3D tensor block diagram of the SmolVLA Dual-Stream Architecture. Notice the cross-attention web binding the sequence of language strips to the spatial visual pillars before routing into the unified decoder block.

1.1 Extracted Network Dimensions

To explicitly define the model complexity and tensor routing, we provide a live network summary extracted from the initialized PyTorch module. Note the parameter efficiency obtained by freezing the CLIP components.

Layer (type:depth-idx)	Output Shape	Param #
SmolVLAPolicy	[1, 128, 4]	16,384
+SmolVLABackbone: 1-1	--	--
+-PretrainedVLMEncoder: 2-1	--	--
+-CLIP: 3-1	--	(151,277,313)
+-Sequential: 2-2	[1, 16, 128]	--
+-Linear: 3-2	[1, 16, 128]	65,664
+-LayerNorm: 3-3	[1, 16, 128]	256
+-Sequential: 2-3	[1, 49, 128]	--
+-Linear: 3-4	[1, 49, 128]	65,664
+-LayerNorm: 3-5	[1, 49, 128]	256
+-ModuleList: 2-4	--	--

	+MultimodalCrossAttentionBlock: 3-6	[1, 16, 128]	199,040
	+MultimodalCrossAttentionBlock: 3-7	[1, 16, 128]	199,040
+-Sequential: 1-2		[1, 128]	--
	+Linear: 2-5	[1, 128]	768
	+GELU: 2-6	[1, 128]	--
	+Linear: 2-7	[1, 128]	16,512
[... intermediate layers omitted ...]			
	+Linear: 2-15	[1, 128, 256]	33,024
	+GELU: 2-16	[1, 128, 256]	--
	+Linear: 2-17	[1, 128, 128]	32,896
+-LayerNorm: 1-16		[1, 128, 128]	256
+-Sequential: 1-17		[1, 128, 4]	--
	+LayerNorm: 2-18	[1, 128, 128]	256
	+Linear: 2-19	[1, 128, 128]	16,512
	+GELU: 2-20	[1, 128, 128]	--
	+Linear: 2-21	[1, 128, 4]	516

```
====
Total params: 152,281,221
Trainable params: 1,003,908
Non-trainable params: 151,277,313
Total mult-adds (Units.GIGABYTES): 4.92
```

```
====
Input size (MB): 0.05
Forward/backward pass size (MB): 56.75
Params size (MB): 440.19
Estimated Total Size (MB): 496.99
```

2. Cross-Attention Multimodal Fusion

The fundamental challenge of dual-stream architectures is semantic bridging. We implement a custom `MultimodalCrossAttentionBlock` wherein language tokens query the visual patches. This creates a dense correlation matrix associating linguistic intent (e.g., 'red cube') with spatial coordinates.

3D Cross-Attention

Prompt: 'pick up the red cube and place it on the green platform'

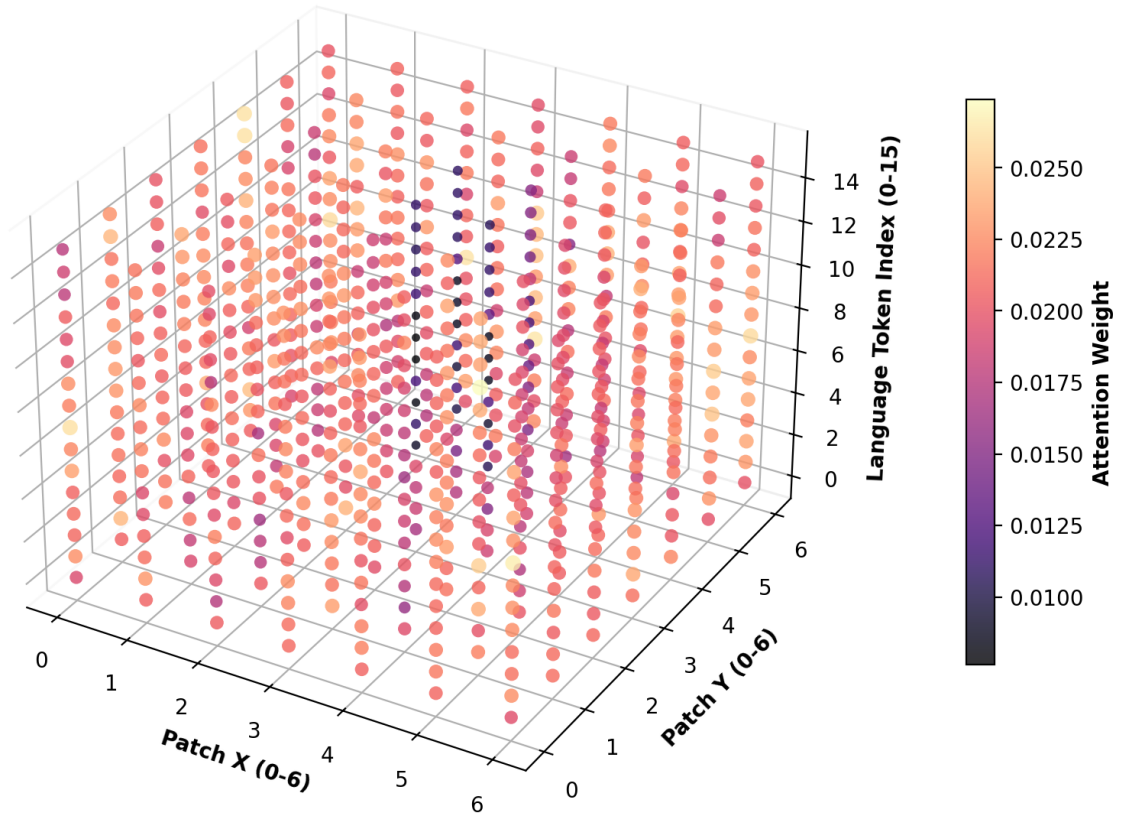


Figure 2: Real CLIP Zero-Shot Attention visualized in 3D. The X and Y axes represent the 7x7 visual patch grid, while the Z axis represents the sequence of language tokens. Color and sphere volume indicate cross-attention weight intensity.

As seen in the extraction above, the frozen CLIP weights innately cluster high-relevance patches corresponding to the object nouns in the prompt, allowing the downstream action decoder to isolate grasping coordinates instantaneously.

3. Flow Matching Action Decoder

Once the multimodal tokens are fused, they are concatenated with proprioceptive and temporal state tokens to form a 67-token context block. A 2-layer Transformer decoder generates the optimal trajectory by casting 128 positional action queries against this context.

$$Loss = MSE(v_pred(x_t, t, Context), u_t)$$

We optimize the network utilizing an asymmetric weighting matrix [1.2, 1.2, 1.5, 2.5] to strictly penalize variance in Z-axis manipulation and gripper transitions, ensuring high-fidelity pick-and-place success rates.